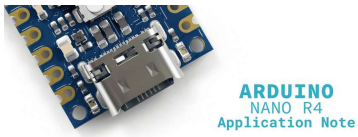
Motor Anomaly Detection with the Nano R4

This application note describes how to implement a motor anomaly detection system using the Nano R4 board, an accelerometer and Edge Impulse.

Author • José Bagur Last revision • 09/10/2025

Introduction

Motor condition monitoring is essential in industrial settings where unexpected equipment failures can cause significant downtime and high maintenance costs. This application note shows how to build a motor anomaly detection system using the Nano R4 board, an accelerometer and Edge Impulse®.



The developed system monitors vibration patterns in real-time to identify unusual operating conditions that may signal mechanical problems, wear or potential failures. The system uses machine learning to detect vibration patterns that differ from normal motor operation, enabling predictive maintenance.

Goals

This application note will help you to:

- Build a motor anomaly detection system that monitors vibration patterns in real-time using an

Collect and analyze vibration data from motors to create baseline patterns and detect changes.

Train a machine learning model using Edge Impulse to detect anomalies based on vibration data.

Deploy the trained model directly to the Nano R4 board for real-time anomaly detection without needing cloud connectivity.

Set up visual feedback through the board's built-in LED to show detected anomalies and system status.

Create industrial predictive maintenance solutions using cost-effective embedded intelligence.

Hardware and Software Requirements

Hardware Requirements

- **Arduino Nano R4** (x1)
- **ADXL335 accelerometer breakout board** (x1) or **Modulino Movement** (x1)
- **USB-C® cable** (x1)
- **Qwiic cable** (x1) (only required if using Modulino Movement option)
- Breadboard and jumper wires (x1 set) (only required if using ADXL335 option)
- Motor or rotating equipment for testing (for example, a small DC motor or fan) (x1)
- Power supply for the motor (if needed) (x1)

Software Requirements

- **Arduino IDE 2.0+** or **Arduino Web Editor**
- **Arduino Renesas Core** (needed for the Nano R4 board)
- **Modulino library** (only required if using Modulino Movement option)

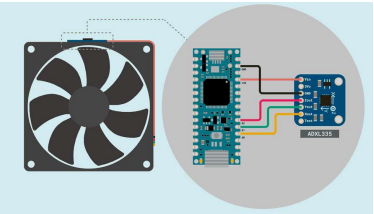
Edge Impulse CLI tools for data collection

The Nano R4 board offers improved processing power with a 32-bit Renesas RA4M1 microcontroller running at 48 MHz, 256 kB Flash memory and 32 kB SRAM for machine learning tasks. For complete hardware specifications, see the Arduino Nano R4 documentation.

Hardware Setup Overview

The electrical connections for the motor anomaly detection system are shown in the diagrams below. Choose the setup that matches your selected accelerometer option.

1. Option 1: ADXL335 Accelerometer Setup

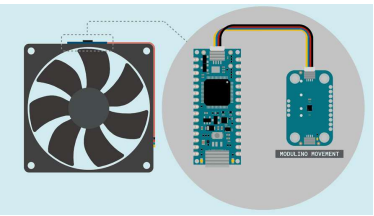


The motor anomaly detection system hardware setup with the ADXL335

This diagram shows the system components using the ADXL335 accelerometer. **The Nano R4 board acts as the main controller**, while **the ADXL335 accelerometer collects vibration data** from the motor through direct analog connections.

The ADXL335 accelerometer connects to the Nano R4 board using analog input pins. In this setup, an external +5 VDC power supply powers both the Nano R4 and the test motor, while the ADXL335 gets power through its built-in voltage regulator from

2. Option 2: Modulino Movement Setup



The motor anomaly detection system hardware setup with the Modulino Movement

This diagram shows the system components using the Modulino Movement. **The Nano R4 board acts as the main controller**, while **the Modulino Movement collects vibration data** from the motor through I²C digital communication via the Qwiic connector.

The Modulino Movement connects to the Nano R4 board using a Qwiic cable for plug-and-play connectivity. The Modulino operates at +3.3 VDC and communicates digitally, providing higher precision measurements and additional features like gyroscope data.

> **Important note**: This power setup is for testing and demonstration only. In real industrial environments, proper power system design should include electrical isolation, noise filtering, surge protection and compliance with industrial safety standards for your specific application.

Circuit Connections

1. ADXL335 Accelerometer Connections

The following connections establish the interface between the Nano R4 board and the ADXL335 accelerometer:

ADXL335 Pin	Nano R4 Pin	Description
VCC	5V	Power supply

ADXL335 Pin	Nano R4 Pin	Description
`X`	`A0`	X-axis analog output
`Y`	`A1`	Y-axis analog output
`Z`	`A2`	Z-axis analog output
`ST`	Not connected	Self-test (optional)

2. Modulino Movement Connections

The following connections establish the interface between the Nano R4 board and the Modulino Movement:

Modulino Movement	Nano R4 Pin	Description
`VCC`	`3V3`	Power supply
`GND`	`GND`	Ground reference
`SDA`	`A4`	I²C data line (via Qwiic)
`SCL`	`A5`	I²C clock line (via Qwiic)

The Modulino Movement connects via Qwiic cable to the Nano R4's onboard Qwiic connector, eliminating breadboard wiring. The default I²C address is `0x6A`.

Physical Mounting Considerations

Proper accelerometer mounting is essential for effective vibration monitoring. The sensor must be securely attached to the motor housing or equipment using appropriate mechanical fasteners. Good mechanical connection between the mounting surface and sensor ensures accurate vibration transmission and reliable

For this application note, we will use a computer cooling fan to simulate motor operation and demonstrate the anomaly detection system. Either the ADXL335 accelerometer or the Modulino Movement can be mounted on top of the fan using a custom 3D-printed enclosure, providing a stable and consistent mounting solution for testing.

Understanding Motor Vibration Analysis

Motor vibrations contain valuable information about the mechanical condition of the equipment. Normal motor operation produces characteristic vibration patterns that stay relatively consistent during healthy operation. Abnormal conditions show up as changes in vibration amplitude, frequency content or timing patterns.

Common Motor Faults

Common motor faults that can be detected through vibration analysis include:

- **Bearing wear**: Creates higher frequency components and increased vibration levels across all axes
- **Misalignment**: Produces specific frequency patterns related to rotational speed, typically appearing in radial directions
- **Imbalance**: Results in increased vibration at the main rotational frequency, primarily in radial directions
- **Looseness**: Causes widespread increases in vibration across multiple frequencies and directions
- **Electrical issues**: May create vibrations at twice the line frequency due to magnetic field changes

The Role of Sensors in Vibration

Effective motor condition monitoring relies on sensors that can accurately detect and measure mechanical vibrations. These sensors must capture the small changes in vibration patterns that indicate developing faults or abnormal operating conditions. The choice of sensor technology directly affects the system's ability to detect problems early and provide reliable maintenance information.

This application note supports two accelerometer options, each offering distinct advantages for motor vibration monitoring applications.

1. ADXL335 Analog Accelerometer

The ADXL335 accelerometer provides accurate, real-time measurements of motor vibrations with several key advantages:

- **3-Axis measurement**: Captures vibrations in X, Y, and Z directions
- **Analog output**: Direct compatibility with the Nano R4 board analog inputs without needing complex digital interfaces
- **Low power consumption**: Only 320 µA current consumption makes it suitable for continuous monitoring applications
- **Wide frequency range**: 0.5 Hz to 1600 Hz bandwidth covers most common motor fault frequencies

The ADXL335 technical specifications include:

Specification	Value	Notes
Measurement range	±3g on all axes	Appropriate for common motor vibration levels
Sensitivity	330 mV/g nominal	Provides good resolution for vibration analysis
Output	Ratiometric analog	+1.65 VDC represents 0g

Specification	Value	Notes
Power supply	+1.8 to +3.6 VDC	Usually regulated to +3.3 VDC on breakout boards

2. Modulino Movement (LSM6DSOXTR)

The Modulino Movement features the LSM6DSOXTR 6-axis inertial measurement unit, offering advanced capabilities for vibration monitoring:

- **6-Axis measurement**: 3-axis accelerometer + 3-axis gyroscope for comprehensive motion analysis
- **Digital I²C interface**: High-precision 16-bit digital output with built-in noise filtering
- **Configurable ranges**: ±2g, ±4g, ±8g, ±16g selectable ranges for different vibration amplitudes
- **High sampling rate**: Up to 6.7 kHz sampling rate for detailed frequency analysis
- **Low power operation**: Advanced power management with multiple operating modes

The Modulino Movement technical specifications include:

Specification	Value	Notes
Measurement range	±2g, ±4g, ±8g, ±16g (configurable)	Adaptable to different motor vibration levels
Resolution	16-bit	Superior precision compared to analog systems
Data rate	Up to 6.7 kHz	High-frequency vibration analysis capable

Specification	Value	Notes
Communication	I²C digital	Immune to electrical noise and interference
Power supply	+3.3 VDC	Low power consumption with sleep modes

3. Sensor Selection Considerations

Choose the ADXL335 if you prefer:

- Simple analog signal processing
- Direct connection without complex protocols
- Cost-effective solution for basic vibration monitoring

Choose the Modulino Movement if you need:

- Higher precision and configurable measurement ranges
- Additional gyroscope data for advanced motion analysis
- Digital noise immunity and built-in filtering
- Professional-grade applications requiring maximum accuracy

Signal processing considerations include selecting appropriate sampling rates based on the expected frequency content of motor vibrations, typically following the Nyquist rule to avoid aliasing. The Nano R4 board's 14-bit ADC provides sufficient resolution for analog sensors, while the Modulino Movement's 16-bit digital output offers even higher precision for demanding applications.

Simple Vibration Monitor Example Sketch

Now that we have covered the hardware

vibration data collection. Before implementing intelligent anomaly detection, we need to collect training data representing normal motor operation for Edge Impulse, a platform that simplifies embedded AI development.

Edge Impulse needs training data in a specific format to build effective anomaly detection models. Our data collection sketch formats the accelerometer readings so Edge Impulse can analyze normal operation patterns and create a model that identifies when new data differs from these patterns.

This section breaks down the example sketch and guides you through its functionality. We will explore how the accelerometer is initialized, how vibration data is collected at consistent intervals, and how the results are formatted for Edge Impulse data collection and model training.

> This example sketch supports both the ADXL335 and Modulino Movement options. Simply uncomment the sensor definition that matches your hardware setup.

The complete example sketch is shown below.

```
/**
  Motor Vibration Data Colle
  Name: motor_vibration_coll
  Purpose: This sketch reads
  accelerometer or Modulino
  The data is formatted for

  @version 2.5 01/06/25
  @author Arduino Product Ex
*/

// Sensor selection - uncomm
#define USE_ADXL335     // F
// #define USE_MODULINO    /

#ifdef USE_MODULINO
  #include <Modulino.h>
  ModulinoMovement movement;
#endif

#ifdef USE_ADXL335
  // Pin definitions for ADX
  const int xPin = A0;
  const int yPin = A1;
  const int zPin = A2;

  // Arduino ADC specificati
  const int ADCMaxVal = 1638
```

The following sections will help you understand the main components of the example sketch, which can be divided into the following areas:

- Sensor selection and initialization
- Hardware configuration and calibration
- Data collection timing and control
- Signal processing and conversion
- Edge Impulse data formatting

Sensor Selection and Initialization

The sketch begins by allowing you to choose which sensor you're using through preprocessor definitions:

```
// Sensor selection - uncomment
#define USE_ADXL335     // For
// #define USE_MODULINO    // F

#ifdef USE_MODULINO
  #include <Modulino.h>
  ModulinoMovement movement;
#endif
```

In this code:

- Only one sensor definition should be uncommented at a time
- The appropriate libraries are included based on your sensor selection
- Sensor objects are instantiated only when needed, reducing memory usage

Hardware Configuration and Calibration

Before we can collect vibration data, we need to configure the Nano R4 board to interface with the ADXL335 accelerometer and set its calibration parameters.

ADXL335 Configuration

For the ADXL335 option, the configuration includes analog pin assignments and calibration parameters:

```
// Pin definitions for ADXL335
const int xPin = A0;
const int yPin = A1;
const int zPin = A2;

// Arduino ADC specifications
const int ADCMaxVal = 16383;
const float mVMaxVal = 5000.0;

// ADXL335 specifications (cal
const float supplyMidPointmV =
const float mVperg = 303.0;

// Conversion factor from ADC
const float mVPerADC = mVMaxVa
```

Pin assignments map each accelerometer axis to specific analog inputs on the Nano R4 board

ADC specifications define the 14-bit resolution and 5V reference voltage used by the Nano R4

ADXL335 specifications use calibrated values measured from the actual breakout board

The conversion factor translates raw ADC readings to millivolts for processing

Important note: ADXL335 breakout boards typically include onboard voltage regulators that convert the input voltage (+5 VDC from the Nano R4) to a lower voltage (usually +3.3 VDC or less) to power the accelerometer chip. This means the actual supply voltage to the ADXL335 may be different from what you expect. The values shown in this code (supplyMidPointmV = 1237.0 and mVperg = 303.0) are calibrated for a specific breakout board and may need adjustment for your hardware.

Modulino Movement Configuration

For the Modulino Movement option, the configuration uses digital I²C communication:

```
#ifdef USE_MODULINO
  // Initialize Modulino I2C c
  Modulino.begin();

  // Detect and connect to mov
  if (!movement.begin()) {
    Serial.println("Failed to
    while (1);
  }

  Serial.println("- Motor Vibr
  Serial.println("- LSM6DSOXTR
#endif
```

In this code:

- `movement.begin()` specifically initializes the Movement sensor with default settings
- No pin assignments are needed as the sensor communicates via I²C through the Qwiic connector
- The sensor automatically configures itself with optimal settings for vibration monitoring
- Values are returned directly in **g** units without requiring additional conversion

Data Collection Timing and Control

To ensure accurate vibration analysis and successful machine learning training, we need consistent data collection timing. These parameters control how data is gathered:

```
// Sampling parameters
const int sampleRate = 100;
const unsigned long sampleTime
```

In this code:

- Sample rate of 100 Hz captures enough frequency response for detecting most motor faults
- Sample time calculation automatically determines the precise timing needed between measurements
- The system streams data continuously without requiring manual start/stop commands

Signal Processing and Conversion

Once we have the raw sensor readings, we need to convert them into useful acceleration values. The conversion process differs between the two sensor options:

ADXL335 Signal Processing

calibrated acceleration data:

```
#ifdef USE_ADXL335
  // Read raw ADC values
  int xRaw = analogRead(xPin);
  int yRaw = analogRead(yPin);
  int zRaw = analogRead(zPin);

  // Convert ADC values to mil
  float xVoltmV = xRaw * mVPer
  float yVoltmV = yRaw * mVPer
  float zVoltmV = zRaw * mVPer

  // Convert to acceleration i
  xAccel = (xVoltmV - supplyMi
  yAccel = (yVoltmV - supplyMi
  zAccel = (zVoltmV - supplyMi
#endif
```

In this code:

- Timing control maintains consistent sample intervals, which is essential for proper frequency analysis
- ADC conversion uses the Nano R4's 14-bit resolution to convert raw readings to millivolts
- Acceleration calculation applies the calibrated zero-point and sensitivity values to get accurate g-force measurements

Important note: For accurate measurements, you should calibrate your specific ADXL335 breakout board by placing it flat on a table with the Z-axis pointing up and measuring the actual voltage outputs. The X and Y axes should read approximately the same voltage (this is your zero-g bias point), while the Z-axis should read higher due to gravity (1g acceleration). The difference between Z-axis voltage and the zero-g bias point gives you the sensitivity in mV/g. This calibration accounts for variations in onboard regulators and manufacturing tolerances.

Modulino Movement Signal Processing

sensor:

```
#ifdef USE_MODULINO
  // Read new movement data fro
  movement.update();

  // Get acceleration values (a
  xAccel = movement.getX();
  yAccel = movement.getY();
  zAccel = movement.getZ();
#endif
```

In this code:

- `movement.update()` refreshes the sensor data
- The acceleration values are returned directly in **g** units, eliminating manual calibration
- Built-in digital filtering provides clean, noise-free measurements
- No conversion calculations are needed as the sensor handles all processing internally

Edge Impulse Data Formatting

The final step formats our acceleration data so it can be used with Edge Impulse data collection tools:

```
// Output CSV format for Edge I
    if (dataValid) {
      Serial.print(xAccel, 4);
      Serial.print(",");
      Serial.print(yAccel, 4);
      Serial.print(",");
      Serial.println(zAccel, 4)
    }
```

In this code:

- CSV format with four decimal places gives us the precision needed for machine learning training
- Single-line output per sample makes it

Comma separation follows standard CSV format that most data processing tools expect

The output format is identical regardless of which sensor is used, ensuring compatibility with Edge Impulse

After uploading the example sketch to the Nano R4 board, you should see output in the Arduino IDE's Serial Monitor similar to this:

Example sketch output showing vibration data collection

Complete Example Sketch

Download the complete data collection example sketch **here**.

Connecting the Vibration Monitor to Edge Impulse

As vibration-based condition monitoring becomes more important for predictive maintenance, connecting our data collection system to Edge Impulse enables the development of intelligent anomaly detection models. Edge Impulse provides a complete platform for embedded machine learning, allowing us to transform raw vibration data into useful insights about motor health.

In this section. we will connect the vibration

models and deploy intelligent anomaly detection directly to the Nano R4 board. This connection transforms our simple vibration monitor into an intelligent system that can detect motor anomalies without needing cloud connectivity.

> If you are new to Edge Impulse, please check out **this tutorial** for an introduction to the platform.

Setting up Edge Impulse Account and Project

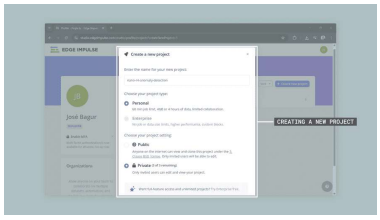
The first step involves creating an Edge Impulse account and setting up a new project for motor anomaly detection. These steps establish the foundation for machine learning model development:

1. **Create Account**: Register for a free Edge Impulse account at studio.edgeimpulse.com
2. **New Project**: Create a new project with the following settings:

   Enter a project name (for example, "`nano-r4-anomaly-detection`")

   Choose project type: Personal (free tier with 60 min job limit, 4 GB data limit)

   Choose project setting: Private (recommended for this application)



Creating a new project on Edge Impulse

1. **Project Configuration**: Once created, the project will be ready for data collection. Sampling frequency and window settings will be configured later during impulse

Data Collection with Edge Impulse CLI

The Edge Impulse CLI provides tools for streaming data directly from the Arduino to the Edge Impulse platform. This eliminates manual file transfers and enables efficient data collection.

Installing Edge Impulse CLI

Before you can collect data from the Arduino, you need to install the Edge Impulse CLI tools on your computer. The installation process varies depending on your operating system.

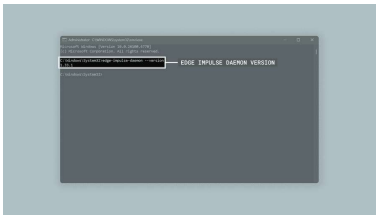
Prerequisites:

- Node.js 14 or higher
- Python 3

For detailed installation instructions specific to your operating system, follow the **official Edge Impulse CLI installation guide**.

Verify the installation with the following command:

```
edge-impulse-daemon --version
```



Edge Impulse Daemon version

Setting up Data Forwarding

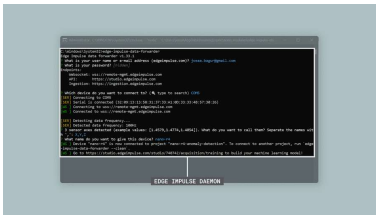
Now that you have the CLI installed, you can set up data forwarding to stream vibration data directly from your Nano R4 board to Edge Impulse.

Connect your Nano R4 to your computer via USB-C cable and upload the data collection sketch. Then open a terminal and run the following command:

```
edge-impulse-data-forwarder
```

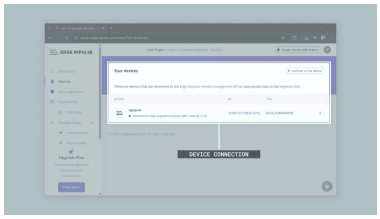
The tool will guide you through the setup process:

1. **Login**: Enter your Edge Impulse username/email and password when prompted
2. **Select Device**: Choose the correct serial port for your Arduino (for example, `COM5`)
3. **Data Detection**: The tool will automatically detect the data frequency (100 Hz) and number of sensor axes (3)
4. **Name Axes**: When asked "What do you want to call them?", enter: `X`, `Y`, `Z`
5. **Device Name**: Give your device a name (for example, `nano-r4`)
6. **Project Connection**: If you have multiple projects, the tool will ask which Edge Impulse project you want to connect the device to. Select your motor anomaly detection project.



Setting up the data forwarder tool

Once configured, the forwarder will stream data from your Nano R4 board to Edge Impulse. You can verify the device connection by checking the "Devices" tab in Edge Impulse Studio. You can then start collecting training data for your machine



Device connection verification to Edge Impulse Studio

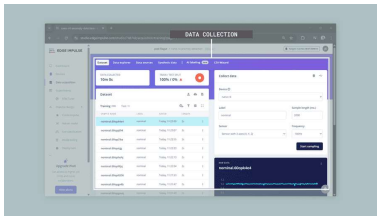
Data Collection Process

With the data forwarder running, you can now collect training data for your anomaly detection model. For effective anomaly detection, you need high-quality data representing normal motor operation in different states.

Start by mounting the accelerometer securely to the motor housing. You will collect two types of normal operation data:

1. **Idle data collection**: With the motor turned off, **collect 10 to 15 minutes of "idle" operation** data through multiple two second windows. This captures the baseline vibration environment without motor operation. Label all data as `idle` in Edge Impulse Studio.
2. **Nominal data collection**: With the motor running under normal operating conditions, **collect 10 to 15 minutes of "nominal" operation** data through multiple two second windows. Vary motor load conditions slightly to capture different normal operating scenarios. Label all data as `nominal` in Edge Impulse Studio.

Edge Impulse can automatically split your collected data into **training (80%) and testing (20%) sets**. The 20 to 30 minutes total of data ensures you have enough



Data collection on Edge Impulse Studio

After data collection, review the collected samples in Edge Impulse Studio for consistency. Check for proper amplitude ranges and no clipping, verify sample rate consistency and timing accuracy and remove any corrupted or unusual samples from the training set.

> **Important note**: The anomaly detection model learns what "normal" looks like from both idle and nominal data. Any future vibration patterns that significantly differ from these learned patterns will be flagged as anomalies. This approach allows the system to detect unknown fault conditions without needing examples of actual motor failures.

Training the Anomaly Detection Model

Once you have collected sufficient `idle` and `nominal` operation data, the next step involves configuring and training the machine learning model for anomaly detection.

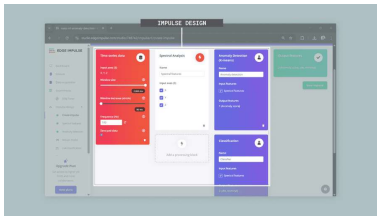
Impulse Design Configuration

Within Edge Impulse Studio, configure the impulse design with appropriate processing and learning blocks. Navigate to the "Impulse design" tab and set up the following blocks:

1. **Input Block**: Configure time series data with window size of 2000 ms,

frequency of 100 Hz to match your data collection sampling rate.

2. **Processing Block**: Add "Spectral Analysis" block for frequency domain feature extraction
3. **Classification Learning Block**: Add "Classification (Keras)" to distinguish between `idle` and `nominal` operating states.
4. **Learning Block**: Select "Anomaly Detection (K-means)" for unsupervised learning approach



Impulse design on Edge Impulse Studio

This dual approach provides a more robust monitoring system where the classifier identifies the current operating state (idle vs nominal) while the anomaly detector flags unusual patterns that don't fit either normal category.

Feature Extraction Configuration

The spectral analysis block extracts relevant features from the raw vibration signals. Configure the following parameters for optimal motor fault detection:

Type: Low-pass filter to focus on motor fault frequencies

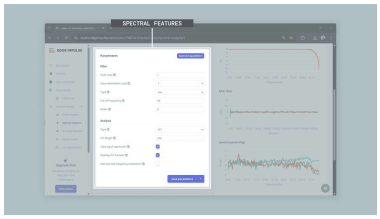
Cut-off frequency: 45 Hz to capture relevant motor vibration characteristics while staying below the Nyquist frequency

Order: 6 for effective filtering

FFT length: 256 points for sufficient frequency resolution

Take log of spectrum: Enable this option to compress the dynamic range of the frequency data

Overlap FFT frames: Enable this option to increase the number of



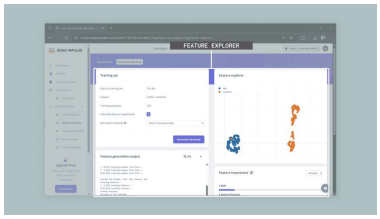
Spectral features on Edge Impulse Studio

Important note: The spectral analysis converts time-domain vibration signals into frequency-domain features. This is crucial because motor faults often appear as specific frequency patterns (like bearing wear creating high-frequency components or imbalance showing up at rotational frequency). The K-means clustering algorithm groups similar frequency patterns together, creating a map of normal operation that can identify when new data doesn't fit the established patterns.

Model Training Process

Follow these steps to train the anomaly detection model using the collected idle and nominal operation data:

1. **Generate Features**: Before clicking "Generate features", enable "Calculate feature importance" to identify which frequency bands are most relevant for distinguishing between idle and nominal states. Then click "Generate features" to extract spectral features from all training data. Edge Impulse will process your data and create the feature vectors needed for training.
2. **Feature Explorer**: Review the feature explorer visualization to verify data quality and feature separation between your idle and nominal classes.



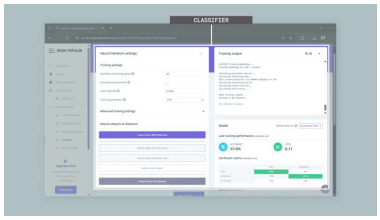
Feature explorer on Edge Impulse Studio

1. **Train Classification Model**:Navigate to the "Classifier" tab and configure the neural network with the following settings:

   Number of training cycles: 30

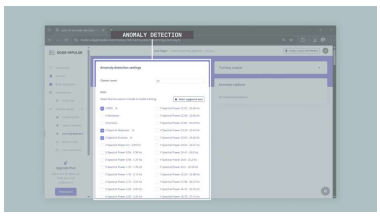
   Learning rate: 0.0005

   Neural network architecture: Configure dense layers with 32 neurons (first layer) and 16 neurons (second layer) to provide good pattern recognition capability for vibration data. Start training and monitor the accuracy metrics.



Classifier on Edge Impulse Studio

1. **Train Anomaly Detection Model**: Navigate to the "Anomaly detection" tab and configure K-means clustering with 32 clusters for pattern recognition. Use "Select suggested axes" to automatically choose the most relevant spectral features based on the calculated feature importance, then start training.



Anomaly detection on Edge Impulse Studio

1. **Model Validation**: Test both models using the validation data to ensure the classifier accurately distinguishes between idle and nominal states, and the anomaly detector properly identifies normal operation patterns.

Important note: The feature explorer shows how well your idle and nominal data separate in the feature space. Good separation means the model can clearly distinguish between different operating states. If the data points overlap significantly, you may need to collect more diverse data or adjust your sensor mounting.

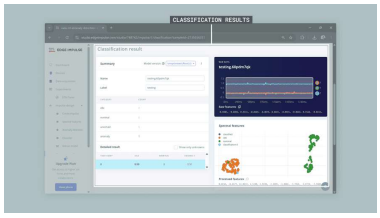
The classification model learns to distinguish between known operating states, while the anomaly detection model creates clusters representing all normal operation patterns. This combination allows the system to both identify the current operating mode and detect unusual conditions that don't fit any normal pattern.

Anomaly explorer on Edge Impulse Studio

Important note: The 32 clusters create a detailed map of normal operation patterns. Each cluster represents a different "type" of normal vibration signature. When new data doesn't fit well into any existing cluster, it's flagged as an anomaly. More clusters provide finer detail but require more training data.

After training completion, validate both model performances using the following methods:

- **Live Classification**: Use the "Live classification" feature of Edge Impulse to test both the classifier and anomaly detector with new motor data to verify their detection capabilities
- **Classification Performance**: Review the confusion matrix and accuracy metrics for the neural network classifier to ensure it properly distinguishes between idle and nominal states
- **Anomaly Detection Analysis**: Review the clustering visualization and anomaly scores to understand how well the model separates normal patterns
- **Threshold Adjustment**: During deployment and real-world testing, you may need to adjust the anomaly detection threshold based on observed false alarm rates and detection sensitivity
- **Validation Testing**: Test both models with known normal conditions to reduce false positives and ensure reliable operation



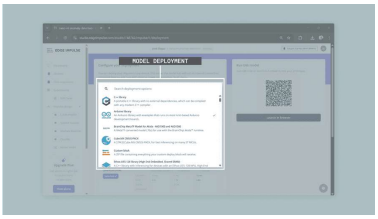
Model validation on Edge Impulse Studio

Important note: The anomaly threshold is critical for system performance. A lower threshold makes the system more sensitive and catches subtle problems but may trigger false alarms. A higher threshold reduces false alarms but might miss early-stage faults.

Model Deployment

After successful training and validation of both models, deploy them as an Arduino library for embedded inference:

1. **Deployment Section**: Navigate to the "Deployment" tab in Edge Impulse Studio
2. **Arduino Library**: Select "Arduino library" as the deployment target
3. **Optimization Settings**: Choose `int8` quantization for memory efficiency on the Nano R4 board
4. **Model Analysis**:Review memory usage and inference timing estimates
5. **Download Library**: Download the generated Arduino library ZIP file
6. **Library Installation**: Install in the Arduino IDE using "Sketch > Include Library > Add .ZIP Library"



Model deployment on Edge Impulse Studio

The generated library includes optimized inference code for both the neural network classifier and K-means anomaly detector, specifically compiled for the Nano R4's Arm® Cortex®-M4 processor. This enables efficient real-time classification of operating states (idle/nominal) and detection of anomalous conditions with low computational overhead.

Improving the Vibration Monitor with Machine Learning

Now that we have trained our machine learning models, we can create a smart

The enhanced system does two things: it identifies whether the motor is running (nominal) or stopped (idle), and it alerts you when it detects unusual vibration patterns that might indicate a problem. This all happens directly on the Nano R4 board without needing an internet connection.

The smart monitoring system can do the following:

- Tell if the motor is running or stopped
- Detect unusual vibrations that might indicate problems
- Flash an LED and show alerts when something seems wrong
- Work continuously without needing internet or cloud services

This enhanced example sketch supports both the ADXL335 and Modulino Movement options. Simply uncomment the sensor definition that matches your hardware setup.

The complete enhanced example sketch is shown below:

```
/**
  Intelligent Motor Anomaly
  Name: motor_anomaly_detect
  Purpose: This sketch imple
  either an ADXL335 accelero
  machine learning model dep

  @version 2.1 01/06/25
  @author Arduino Product Ex
*/

// Include the Edge Impulse
#include <nano-r4-anomaly-de

// Sensor selection - uncomm
#define USE_ADXL335     // F
// #define USE_MODULINO    /

#ifdef USE_MODULINO
  #include <Modulino.h>
  ModulinoMovement movement;
#endif

#ifdef USE_ADXL335
  // Pin definitions for ADX
  const int xPin = A0;
  const int yPin = A1;
  const int zPin = A2;
```

The following sections will help you understand the main components of the enhanced example sketch, which can be divided into the following areas:

- Edge Impulse library integration
- Real-time data collection and buffering
- Machine learning inference execution
- Anomaly detection and alert system

Sensor Selection and Edge Impulse Library Integration

The enhanced sketch starts by selecting the appropriate sensor and including the Edge Impulse library with the necessary constants.

```
// Include the Edge Impulse li
#include <nano-r4-anomaly-dete

// Sensor selection - uncommen
#define USE_ADXL335     // For
// #define USE_MODULINO    //

#ifdef USE_MODULINO
  #include <Modulino.h>
  ModulinoMovement movement;
#endif

// Detection parameters
const float anomalyThreshold =
const float confidenceThreshol

// Data buffer for the models
float features[EI_CLASSIFIER_D
```

The library contains both the classification model (to identify if the motor is idle or running) and the anomaly detection model (to spot unusual vibrations). The sensor selection allows the same code to work with either accelerometer option by simply changing the `#define` statement.

For the ADXL335 option, calibrated constants use measured values from the actual sensor rather than theoretical values for better accuracy:

```
#ifdef USE_ADXL335
  // ADXL335 specifications (ca
  const float supplyMidPointmV
  const float mVperg = 303.0;
#endif
```

For the Modulino Movement option, no calibration constants are needed as the sensor provides direct acceleration values in g units through its digital interface.

Real-Time Data Collection And Buffering

The system collects vibration data for the

data collection process adapts automatically based on your selected sensor.

ADXL335 Data Collection

For the analog ADXL335, the system performs ADC conversion and calibration:

```
#ifdef USE_ADXL335
  // Read raw ADC values
  int xRaw = analogRead(xPin);
  int yRaw = analogRead(yPin);
  int zRaw = analogRead(zPin);

  // Convert ADC values to mil
  float xVoltmV = xRaw * mVPer
  float yVoltmV = yRaw * mVPer
  float zVoltmV = zRaw * mVPer

  // Convert to acceleration i
  xAccel = (xVoltmV - supplyMi
  yAccel = (yVoltmV - supplyMi
  zAccel = (zVoltmV - supplyMi
#endif
```

Modulino Movement Data Collection

For the digital Modulino Movement, the system reads calibrated acceleration values directly:

```
#ifdef USE_MODULINO
  // Read new movement data fro
  movement.update();

  // Get acceleration values (a
  xAccel = movement.getX();
  yAccel = movement.getY();
  zAccel = movement.getZ();
#endif
```

Complete Data Collection Function

The complete `collectVibrationWindow()` function combines both sensor options:

```
void collectVibrationWindow()
  featureIndex = 0;
  unsigned long sampleInterval

  // Collect samples according
  for (int sample = 0; sample
    unsigned long sampleStart
    float xAccel, yAccel, zAcc

    // Sensor-specific data ac
    // ...

    // Store in feature buffer
    features[featureIndex++] =
    features[featureIndex++] =
    features[featureIndex++] =

    // Wait for next sample ti
    while (micros() - sampleSt
      // Precise timing wait
    }
  }
}
```

This function reads vibration data from the selected accelerometer and converts it to the format needed by the machine learning models. It collects exactly 200 samples (two seconds of data) and maintains precise timing to match the training data.

Machine Learning Inference Execution

The system analyzes the collected vibration data using both machine learning models to determine motor state and detect anomalies. The inference process is identical regardless of which sensor you're using, as both provide the same data format to the models.

```
void performInference() {
  ei_impulse_result_t result

  signal_t features_signal;
  features_signal.total_lengt
  features_signal.get_data =

  EI_IMPULSE_ERROR inferenceR

  if (inferenceResult == EI_I
    // Find the best classifi
    String motorState = "unce
    float maxConfidence = 0.0

    for (uint16_t i = 0; i <
      if (result.classificati
        maxConfidence = resul
        motorState = ei_class
      }
    }

    // Check if we're confide
    if (maxConfidence < confi
      motorState = "uncertain
    }

    // Check for anomalies
    bool isAnomaly = (result.
```

This function runs both models on the collected data. The classification model determines if the motor is idle or running, while the anomaly detection model checks if the vibration patterns look unusual. The system only makes a classification if it's confident enough in the result.

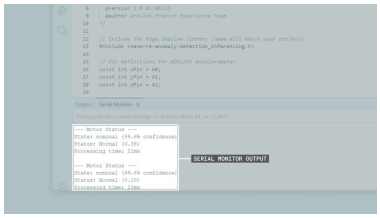
Anomaly Detection and Alert System

The system provides feedback when it detects problems with the motor, using the same alert mechanism regardless of which sensor is providing the data.

```
void triggerAnomalyAlert() {
  unsigned long currentTime =

  if (currentTime - lastAlert
    // Flash LED three times
    for (int i = 0; i < 3; i++
      digitalWrite(alertPin, H
      delay(150);
      digitalWrite(alertPin, L
      delay(150);
    }

    // Log the event
    ei_printf("*** ANOMALY DET

    lastAlert = currentTime;
  }
}
```

When the system detects unusual vibration patterns, it flashes the built-in LED three times and prints a warning message. The alert system prevents spam by waiting at least 2 seconds between alerts, even if multiple anomalies are detected.

After uploading the enhanced sketch to the Nano R4 board, you should see the following output in the Arduino IDE's Serial Monitor during normal operation:



Enhanced example sketch output showing real-time anomaly detection

When an anomaly is detected, the built-in LED will flash three times and the serial output will display the anomaly score above the configured threshold along with a warning message.

Complete Enhanced Example Sketch

The complete intelligent motor anomaly detection sketch can be downloaded here.

System Integration Considerations

When deploying the intelligent anomaly detection system in industrial environments, consider the following factors based on your sensor choice:

- **Environmental Protection**: Protect the Nano R4 board and accelerometer from dust, moisture and temperature extremes using appropriate enclosures rated for the operating environment.
- **Mounting Stability**: Ensure secure mechanical mounting of both the accelerometer sensor and the Nano R4 enclosure to prevent sensor movement that could affect measurement accuracy.
- **Power Management**: Implement appropriate power supply filtering and protection circuits, especially in electrically noisy industrial environments with motor drives and switching equipment.
- **Calibration Procedures**: Establish baseline measurements for each motor installation to account for mounting variations and motor-specific characteristics that may affect anomaly thresholds.
- **Maintenance Integration**: Plan integration with existing maintenance management systems through data logging interfaces or communication protocols for complete predictive maintenance programs.

Conclusions

This application note demonstrates how to implement motor anomaly detection using

combined with Edge Impulse machine learning platform for industrial predictive maintenance applications.

The solution combines the Nano R4's 32-bit processing power with Edge Impulse's machine learning tools to enable real-time anomaly detection directly on the embedded device. This eliminates the need for cloud connectivity and provides immediate response to potential equipment issues with inference times under 20 milliseconds.

The unsupervised anomaly detection approach using K-means clustering requires only normal operation data for training, making it practical for industrial deployment where fault data may be difficult to obtain. This approach can detect previously unseen fault conditions that differ from established normal patterns.

Next Steps

Building upon this foundation, several enhancements can further improve the motor anomaly detection system:

- **Multi-Sensor Fusion**: Integrate additional sensors such as temperature, current or acoustic sensors to provide a more complete view of motor health and improve detection accuracy. The Modulino Movement's built-in gyroscope can provide additional motion analysis capabilities.
- **Wireless Communication**: Add wireless connectivity using LoRaWAN, Wi-Fi or other modules to enable remote monitoring and integration with existing plant systems.
- **Advanced Analysis**: Implement data logging for trend analysis, industrial protocol integration for SCADA systems or multi-class fault classification to distinguish between different types of motor problems.

The foundation provided in this application

specific industrial requirements, with the flexibility to choose the sensor option that best fits your application needs.

Suggest changes

The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here.

Need support?

Help Center
Ask the Arduino Forum
Discover Arduino Discord

License

Was this article helpful?